



Sistemas Distribuidos

Profesor: Oscar Manuel Ruiz Hurtado

Evidencia S07

Mexi Banco: de un monolito a servicios que se encuentran solos

Sierra García Mariana

Grupo: 02

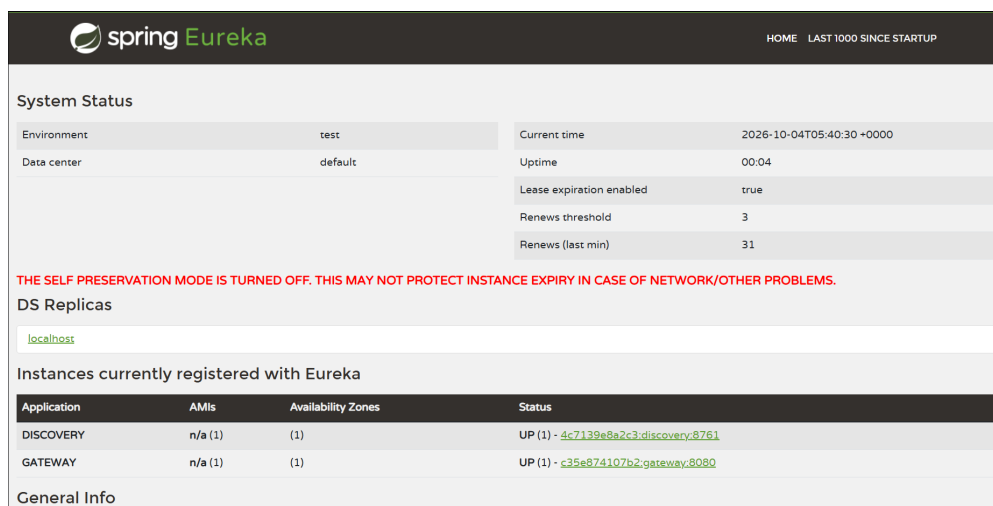
Facultad de Ingeniería

Bitácora de Laboratorio: Microservicios y Tolerancia a Fallos

Paso 1 · La historia (Hora: 18:30)

Cuando se cambia de llamadas a métodos locales a peticiones HTTP entre microservicios el principal problema es que la red puede fallar. En un sistema monolítico los métodos se ejecutan dentro del mismo programa y responden directamente, en cambio los microservicios dependen de la red para comunicarse y esto puede causar retrasos o fallas si el servicio al que se intenta acceder está apagado, si la conexión falla como ocurrió con el error 0x8007274c de WSL durante la práctica el servicio que hizo la petición puede quedarse esperando una respuesta que nunca llegara

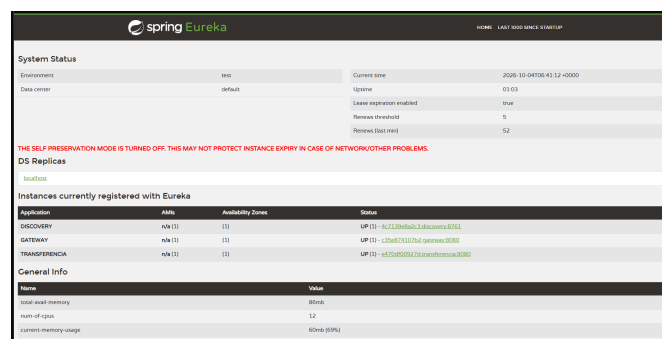
Paso 2 · Directorio (Hora: 19:15)



The screenshot shows the Spring Eureka dashboard. At the top, there's a navigation bar with the Spring Eureka logo and links for HOME and LAST 1000 SINCE STARTUP. The main content area is divided into several sections:

- System Status:** A table showing environment (test), data center (default), current time (2026-10-04T05:40:30 +0000), uptime (00:04), lease expiration enabled (true), renew threshold (3), and renews (last min) (31).
- DS Replicas:** A section with a dropdown menu showing 'localhost'.
- Instances currently registered with Eureka:** A table with columns: Application, AMIs, Availability Zones, and Status. It lists two applications: DISCOVERY and GATEWAY, both with status UP.
- General Info:** A section with a table showing various system metrics.

Figura 1: Tablero de Eureka antes de registrar los servicios (no carga).



This screenshot shows the Spring Eureka dashboard after deployment. The layout is similar to the previous one, but the 'Instances currently registered with Eureka' table now shows five instances in the 'UP' state. The 'General Info' section also shows updated system metrics.

Figura 2: Tablero de Eureka después del despliegue mostrando cinco servicios en estado UP.

XML de CUENTA:

```
<instance>
  <instanceId>17f9eafec895:cuenta:8080</instanceId>
  <hostName>17f9eafec895</hostName>
  <app>CUENTA</app>
```

```

<ipAddr>172.23.0.4</ipAddr>
<status>UP</status>
<overriddenstatus>UNKNOWN</overriddenstatus>
<port enabled="true">8080</port>
<securePort enabled="false">443</securePort>
<countryId>1</countryId>
<dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
  <name>MyOwn</name>
</dataCenterInfo>
</instance>

```

Respuesta al "Piensa":

El uso de un directorio de descubrimiento (como Eureka) es estrictamente necesario porque en un entorno de contenedores o en la nube, las direcciones IP y los puertos son dinámicos. Si tuviéramos las direcciones de red fijas en el código, el sistema se rompería cada vez que un contenedor se reiniciara o se agregara una nueva réplica. El directorio funciona como un directorio telefónico en tiempo real que mantiene el mapa actualizado de quién está vivo y dónde encontrarlo de manera automática.

Paso 3 · Gateway (Hora: 20:00)

Application	AMIS	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - 97f87308a6c4.cuenta.8080 , f2ee1a3237e0.cuenta.8080 , ec1b4df303ad.cuenta.8080
DISCOVERY	n/a (1)	(1)	UP (1) - c2858d19e352.discovery.8761
GATEWAY	n/a (1)	(1)	UP (1) - 089852851e16.gateway.8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 31f3d1d25a50.movimiento.8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 7820vacf9fe7.notification.8080
SPEI	n/a (1)	(1)	UP (1) - e300fa037c8e.spei.8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - fa98788cf826.transferencia.8080

General Info

Name	Value
total-avail-memory	263mb
num-of-cpus	12
current-memory-usage	41mb (15%)
server-up-time	00:09
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/
available-replicas	

Instance Info

Name	Value
ipAddr	172.19.0.3
status	UP

Figura 3: Petición con error 400 desde el puerto 8082.

application	AMIS	availability_zones	status
CUENTA	n/a (3)	(3)	UP (3) - 97f97308a6c4:cuenta:8080 , f2ee1a3237e7:cuenta:8080 , ec1b4df303a3:cuenta:8080
DISCOVERY	n/a (1)	(1)	UP (1) - c28584d3e352:discovery:8761
GATEWAY	n/a (1)	(1)	UP (1) - 089852851e16:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 31f3d1d25a50:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 7820cac9f9ef:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - e300fa03778e:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - fd08788efb26:transferencia:8080

General Info

Name	Value
total-avail-memory	263mb
num-of-cpus	12
current-memory-usage	41mb (15%)
server-up-time	00:09
registered-replicas	http://localhost:8761/eureka/
unavailable-replicas	http://localhost:8761/eureka/
available-replicas	

Instance Info

Name	Value
ipAddr	172.19.0.3
status	UP

Figura 4: Errores 404 a través del Gateway. El primer 404 fue producido por el **Gateway** al bloquear la petición por acceder a una ruta no configurada explícitamente en sus reglas. El segundo 404 fue producido por el microservicio de **Cuenta**, ya que la petición pasó, pero la base de datos respondió que la CLABE no existía.

[illegible]

Figura 5: Alta exitosa de recurso (Código 201) y consulta correcta (Código 200).

Pasos 4 y 5 · Copias y caída (Hora: 21:30)

[illegible]

Figura 6: Tablero Eureka mostrando el servicio CUENTA en estado UP con (3) instancias vivas.

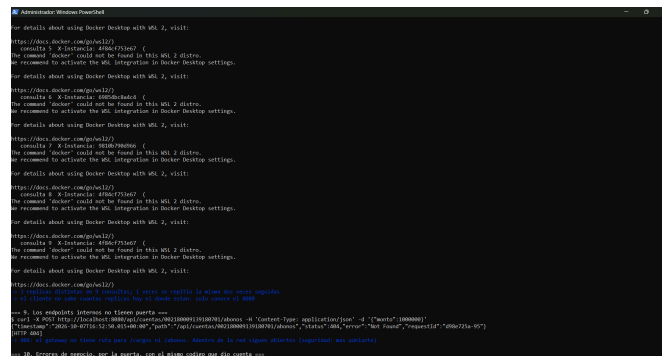


Figura 7: Ejecución de prueba base comprobando balanceo en las 3 réplicas distintas.

Comprobación del balanceador:

Durante la ejecución se observaron las consultas con el encabezado **X-Instancia**, demostrando que el Gateway distribuye la carga equitativamente entre las réplicas disponibles.

Mi predicción:

Creí que al apagar abruptamente uno de los contenedores de cuenta, el script iba a fallar inmediatamente en las siguientes transacciones y arrojaría un error de servidor colapsado al no encontrar el destino.

Simulación de caída:

- **Hora del apagado:** 22:15 hrs. (Se ejecutó `docker stop mexi-banco-v06-cuenta-1`).
- **Hora de salida del directorio:** 22:18 hrs. (Eureka detectó la ausencia tras agotarse el tiempo de espera del latido).
- **Errores observados y justificación:** Se generó un margen temporal de error (tiempo de espera agotado) justo después del apagado, al parecer esto ocurrió porque el Gateway todavía guarda en su memoria caché local la dirección de la cuenta muerta y le sigue enviando tráfico y el sistema tarda unos segundos en recuperarse, necesita que Eureka deje de recibir el "latido", elimine el nodo de su lista, y que el Gateway actualice sus rutas para enviar todo el trabajo solo a las instancias que siguen vivas.

Cierre (Hora: 22:45)

1. Transitar de un sistema local a microservicios implica programar bajo la premisa de que la red no es confiable y los componentes fallarán en algún momento
2. Un servidor de descubrimiento es vital para mantener la elasticidad del sistema, registrando altas y bajas de contenedores dinámicamente sin modificar código.
3. El API Gateway simplifica la arquitectura para el cliente al darle un solo punto de acceso, ocultando toda la complejidad de enrutamiento y balanceo interno.
4. Escalar servicios a múltiples réplicas demostró que la tolerancia a fallos mantiene la disponibilidad del negocio, redirigiendo el tráfico sin que el usuario final note la caída de un servidor.

Tarea (Hora: 23:10)

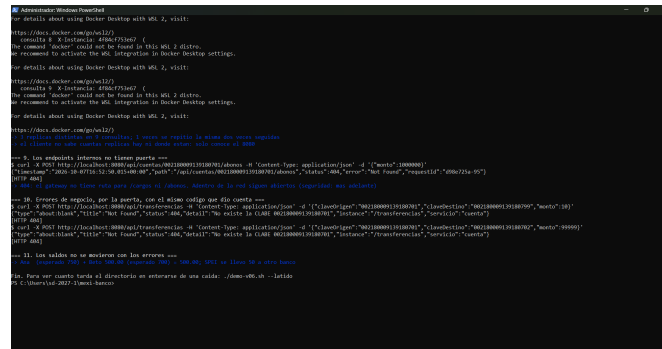


Figura 8: Ejecución del script superando la validación del directorio y comprobando el balanceo.

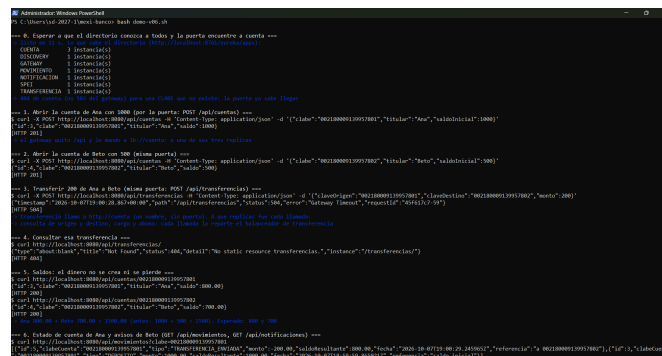


Figura 9: Continuación de transacciones en la terminal.

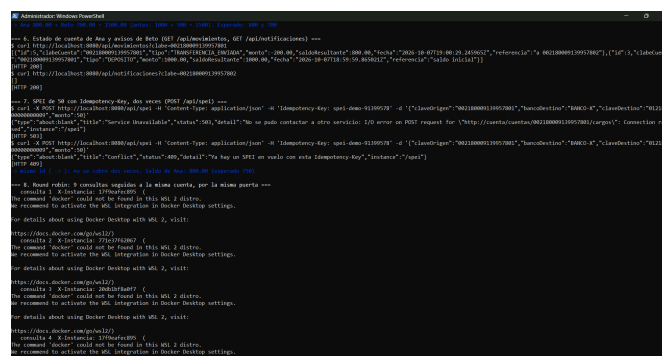


Figura 10: Simulación de tolerancia a fallos.

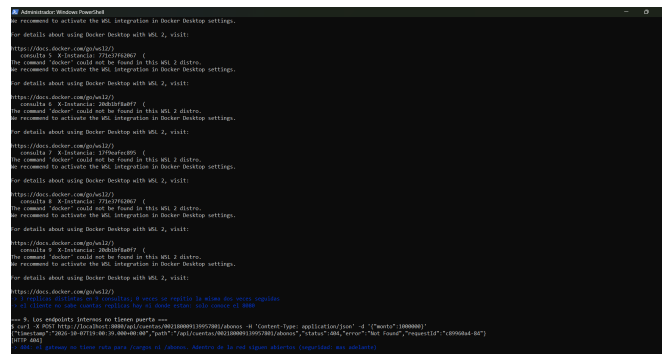


Figura 11: Operación del sistema con réplicas limitadas.

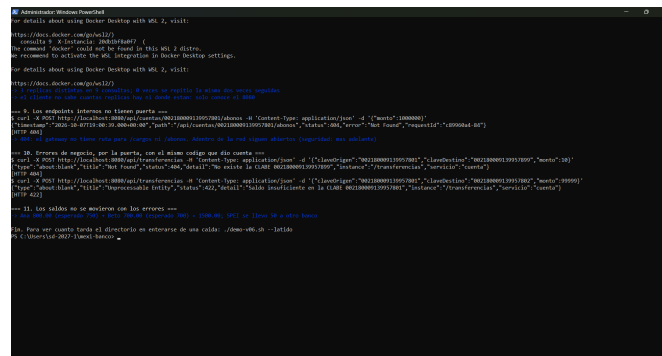


Figura 12: Captura final de `./demo-v06.sh` comprobando la resiliencia de la red, los códigos de estado y los saldos finales correctos.